

Monte Carlo simulation of terpolymerization: Optimization of the computational time

Amanda M. do Amaral¹, Artur S. C. Rego¹, Sheriton R. Valim¹, Joao P. B. Soares²,
Amanda L. T. Brandão¹.

1. Pontifícia Universidade Católica do Rio de Janeiro. 2. University of Alberta.

KEYWORDS: Simulation; Optimization; Monte Carlo; polymerization; Parallelization.

ABSTRACT: The present study analyzes the Monte Carlo simulation of a steady state terpolymerization process to reduce the overall computational time. The most used sorting algorithms were compared to find the best for the polymerization process under study. The size of the vector used to save the polymer chain lengths was investigated and found to influence the simulation time. Parallelization procedures to improve the code efficiency were also considered. Including all these improvements into the code, the time required to run the polymerization process under steady state conditions was reduced to 99,94 % of the original processing time.

1. Introduction

Polymers are macromolecules with long chains which have a high molecular weight. The minor molecular weight monomers repetition joins themselves through covalence bonds to constitute a heavier structure known as polymers.

These macromolecules can be found in the environment (natural) or can be synthetic. The ones in which there are no modifications are presented in most of physical structures existent in the world like living organisms. There are polymeric sources in our DNA, in the amide of proteins, petroleum, and celluloses. In the latter, the carbohydrate is composed of glucose associated with oxygen-generating big chains.¹

The commercial polymers produced using natural composites as raw material could have multiple functions and applicability, such as semiconductors, application on medicine or bulletproof vests.² This latter is full of fortification, which raises people's protection, and it can be composed of polyester, polyamide, or aramid. Lopes et al³ found out that bulletproof vests can also be made of polyethylene fibers with ultra-molecular weight.³ Some polymer classes are applied as pharmaceutical nano-carriers for drugs like for sliding surfaces of artificial joints or suture materials and meshes, helping the biomedical field.⁴

Other type of polymer classification is related to the monomeric composition. A polymer can be a homopolymer or copolymer. For the first case, the structure has just one type of monomer and the second one is composed by two or more different monomer units. The benefit of utilizing copolymers is to fuse properties that exists in an homopolymer and configures it to specific goal.⁵ The present study investigates a terpolymer which is a polymer synthesized from three different monomers and fits in the category of copolymers.

Polymers can also be defined by its distributions of molecular weight (MWD) or chain length distribution (CLD) which is described by Flory Schulz most probable distribution. Other distributions are Chemical composition (CCD) and long-chain branching (LCB). These distributions for multiple-site catalysts are commonly represented as weighted summations of the single-site distributions contained in the polyolefin. The number average molecular weight (M_n) - which is the total mass chain sur the total number of chains - and the weight average molecular weight (M_w) - which is a ponder M_n - are also utilized for polymer classifications.¹

As a polymer it is necessary fulfilling all the prerequisites if it expects to suit a specific industry demand. For example: To be super resistant to impact requires a specific characteristic thorough production control such as high-compression formations that works as springs.² That exemplifies why simulations are important. In fact, process controlling is essential to guarantee the success of the project itself and avoid possible crisis. It makes it possible to predict the polymer's specificity according to the process conditions used in the production.⁶ Since simulations are entirely virtual, the cost for executing it is reduced because it would not be necessary to test and spend raw material in previous trials. Besides that, with less physical tests than not using simulation, that will be a cutback on the time spent to the success of the project.⁷ Because the cost of a polymer is directly connected with the polymerization process's difficulty and the monomer availability in nature.¹

The simulation enables a massive variety of products to attain our requests as a society, with reduced cost, which incites the industry to create more specific and unique polymers to whichever proposed functionality.

Diverse models can be used to simulate polymeric processes, divided into deterministic, stochastic and hybrid models.⁸

The deterministic models solve a system of equations that describe the whole mass and energy balances, but, when applied to simulate polymerization reactions, the deterministic models become more complex since it creates a system with infinite differential equations, requiring the adoption of hypotheses to become solvable. That would increase the complexity of the problem in the present study. One of the methods that belong to this category is the standard method of moment, being commonly used in kinetic studies. The orthogonal collocation method is another deterministic method very used a hardened time to simulate polymerizations.^{9,10,11,12,13} This technique utilizes polynomial functions as approximation to the system-dependent variables, turning them independent and consequently reducing the complexity of its solution. However, it does not give us the MWD without assumptions. These suppositions made for deterministic models can deviate the simulation from what really occurs when generating a polymer. Other deterministic method is the numerical inversion of probability generating functions, which consists in transforming the full MWD into the generating function domain.¹⁴ The equations are solved and then numerically inverted to the chain length at discrete points. The technique has been used to model different polymerizations^{15,16,17,18,19} but its drawback are the oscillations in MWD for complex cases.¹⁴ The importance of the polymer properties, like MWD, to validate the study is critical. It turns able the visualization of the distribution onwards the polymer chain. Through this, it is possible to cross the studied case with the simulation made, what leads into a model validation. Since the MWD is desirable to confirm the case study model, the use of these deterministic methodologies is not recommended. It could impact on the polymer properties, leading to an undesired result.²⁰

On the other hand, the Monte Carlo method (MC), a stochastic method, works well with complex systems because it utilizes simple logical sequences. The MC takes advantage of a computational orientation to simulate the process and results with no analytical solution.²¹ Beside the fact of being a model free of the imposition of solving ordinary differential equations (ODEs) - that could be nonlinear -, pending logic only, it is also widely employed to copolymerization.²² Its application is perennial justified because the polymeric chains could be simulated individually during the process, giving rise to assertiveness and precision on the model predictions represented by the implemented code.²³ The MC method is suitable for solving random distribution problems but it often requires a large amount of calculations what lead it to a time-consuming simulation.²⁴ That is why an optimization need to be enslavement, to assure the efficiency of the code.^{20,21,25,26}

In fact, the MC method was used extensively by many authors.^{27,28,29,30,31} Monte Carlo applied to a polymerization problem was before implemented by Özçam and Teymour²⁵, whose proposed a hybrid method for modeling polymerization named “Chain-by-Chain Monte Carlo” method (CBC-MC) conducting an essentially stochastic simulation while obtaining some of the information from deterministic calculations through differential moment equations. The study utilizes two different chemistries: Nitroxide-mediated polymerization (NMP) and atom transfer radical polymerization (ATRP) and achieved significantly reduces in simulation time when comparing using only MC, especially because it predicts full information regarding the microstructure of chains.²⁵ Yousef et al²⁶ on the other hand, explored it differently, mentioning the algorithm as an engineering resource for inverse polymer’s simulation. They utilized the kinetic MC (KMC) method allied with artificial intelligence techniques that received the target microstructural characteristics of polymer chains and predict the input operating factors. Nie et al³² and Lai and Binder³³ utilized the method in their research about one same subject: grafted polymers. Respectively, stereo complex crystallites (SCs) and polymer brushes. Their study brings more information about the behavior and formation of grafted systems. The First research culminates in a new way to improve the stereo complex formation in poly(L-lactic acid) (PLLA) and poly(D-lactic acid) (PDLA) blend.³² While the second results shown that polymer chains on lattices and its properties can be understood in terms of appropriate scaling concepts.³³

Based on the terpolymer example on chapter 6 (Polyolefin Microstructural Modeling) at Soares and McKenna³⁴, the present study is a simulation of a theoretical mechanism of an olefin bulk terpolymerization - no solvents - at steady state using the MC method. The steady state occurs when the same conditions are used from the beginning of the program until its end. During the simulation, when the reaction to grow the polymer chain

is selected, there are three available monomers to be inserted into this chain. Each monomer's choice has the same probability, and they are independent and not influenced by what was previously formed in the chain. All predictions have equal chances to happen and not a single part of what was already generated **impacts** because no conditions are modified. Otherwise, in the dynamic state, each event probability is updated during the whole simulation, since the number of molecules in the system are varying and, consequently, affects the reaction rate expressions .³⁴

Many MC simulations at steady state were previously made to polyolefins (polymer with an olefin monomer on its conformation C_nH_{2n}). Ramos et al³⁵ utilized short chain branched polyolefins as model of metallocene ethylene/ α -olefin copolymers simulated by MC while Nathanael et al³⁶ applied the method for polyolefins made with two metallocene catalysts in a batch reactor. The development of this type of simulation was directly led by the commercial operations started happening in continuum reactors. It permits ~~that~~ the computational field **could** work with the same premise of the polymers industry.³⁷

Zhao et al³⁸ reported a polymer model's reproduction with hives that look like a tree was recreated, using the MC method.³⁹ This operation was simulated in a steady-state to meet the prerequisites of a CSTR reactor. The research was successful since its results agreed with the deterministic predictions obtained utilizing the method of moments. When comparing both the method of moments and the MC, either was not behind and presented excellent conclusions showing that it is suitable when applying with steady-state to simulate some polymerizations. Szymanski et Sosnowski⁴⁰, Rego et Brandao⁴¹ and Pournemat et al⁴² also utilized the steady-state allied with MC method in polymer's simulations. Beyond that, its applicability is broad since the proposed algorithm is not complex **and** generally easier to implement when compared with other model approaches like deterministic ones. ~~The~~ modeling using the MC method **was presented** even more relevant **facing** others because it can predict the polymer molecular weight distribution.³⁹

2. Monte Carlo Simulation Procedure

As the MC method is a stochastic algorithm, each simulation returns a distinct output answer, but all the answers are within a certain interval. At the end of simulation, we have an array that each position has the number of polymer chains formed with length equals to the current array position. Additionally, we know how many monomers types each polymer chain has. To **generates** the final MWD, it is necessary to use sorting algorithms which were not discussed by any study yet as far we have knowledge. The computational time spent in this step is a factor that needs to be added to the full program cost.

Figure 1 shows an example of the MC method's functioning on a simple use to calculate how many times an even number appears when rolling a dice for N plays. The method uses random numbers that follow the uniform distribution to simulate what number it will get from the round when playing dice, and then it deliberates the probability as a result.



By observing the example on Figure 1, it can be noticed that less the N (number of dice rolled) introduced in the calculations, more twisted from reality this resulting probability will be. That also happens for any other prediction made utilizing Monte Carlo. As a stochastic method, as more significant the number of code iterations, the more faithful to the correct final result of the problem in question. In other words, it is viable to reproduce the model with high proximity from the real occurrences, but it will reflect in the computational cost.

The accuracy exists; however, there is a high computational accomplishment behind since that to create polymerizations, the monomers chains are built one by one until the moment it stops growth.⁴³ This long time spent simulating with efficiency is no doubt the great barrier to be overtaken to achieve the chosen study under consideration.

Among the different forms to accelerate the simulation process, one of them is to parallelize. This strategy consists of executing a determinate sequel at the same time for many counts. This is showed in Figure 2, where the flowchart X corresponds to the generation and finalization chain process. In other words, some polymeric chains are simultaneously created while others could be ending. For example, imagine that it was made by only one person and now it is split between others (in this case, other three). Naturally, the objective will be achieved faster than before. At the end of parallelization, all the occurrences get together to compose one singular polymer. Thus, the time reduces in comparison with the serial execution of the process – without this machination.

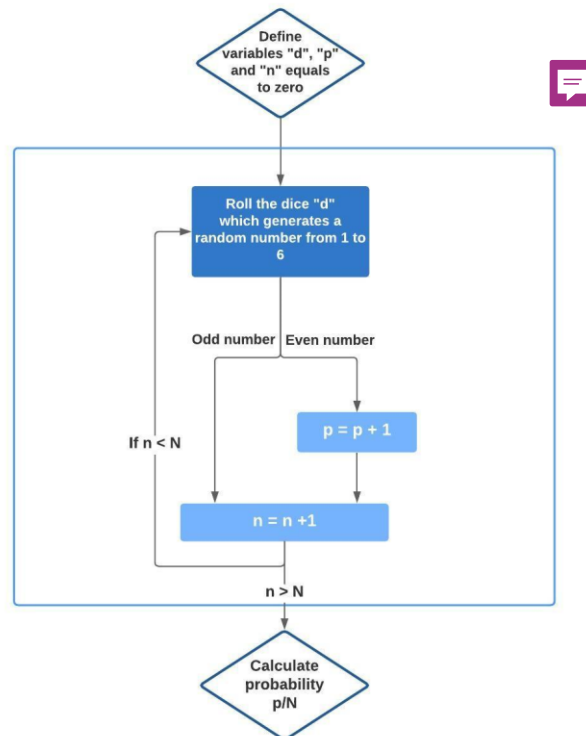


Figure 1: Even MC Probability

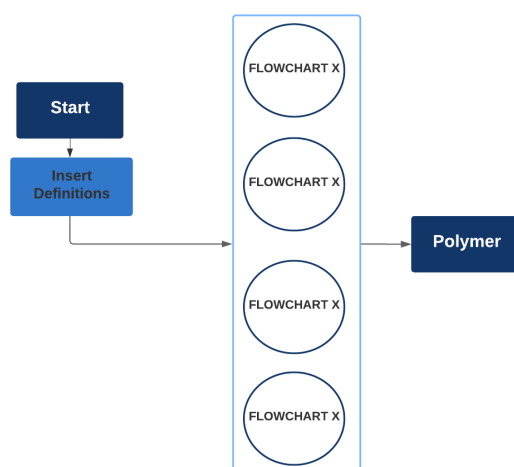


Figure 2: Scheme of a Parallelization strategy

Nowadays parallelization techniques applied to polymerization simulation with Monte Carlo are the focus of many researches in literature.²³ One of the publications uses the same study treated here. The proposal is **the same even**, so the methodology **distinguish** since the study **makes modeling in** GPU, a microprocessor that shapes its modeling with graphic association rendering from 2D and 3D. Aside from it, a **CUDA** which is a parallel computing platform and application programming interface **is** applied. This tool is **appropriated** in applications on GPU for its better efficiency comparing to CPU processors. Another item to highlight is a graphic produced from the molecular weight distribution (MWD) versus chain length distribution (CLD) which **was not previously discussed the step by step of how the figure was obtained**. The sorting method and its time spent also was not explored by the author.²³

As aforementioned, the present work simulates a terpolymerization using Monte Carlo model at steady state founded on previously mentioned case presented in the book of Soares and McKenna³⁴. Best strategies were evaluated and identified through an analytical path accompanying to reduce the computational cost which is high due to the MC model used. The strategies that were used for the present study are **composed by** implementations to accelerate the simulation, such as parallelization and other algorithms that were **addicted in** the program **chasing** a rapid sorting. It **is the foremost peak** that the sorting time has a significant contribution to the process's final computational cost.

2. Python Programming Language

Python language was the chosen one, among others, to bring into practical action **to** the polymer's simulation code, due to its simple learning and for being open source. Utilizing this written machination for coding, techniques as script reading code in a language with rapid processing, use of parallel subterfuge or polymeric chain terminated and sorting executions with different algorithms will be applied to optimize the process.

The priority for this work was to carefully choose a language that could be used by everyone. An open-source code, simple to learn, assimilate, apply and modify (if necessary). Compatible with major platforms and systems, Python was the language elected for this study.

3. Methodology

All simulations were run with a personal microcomputer with following configurations: Intel Core i7-10510U CPU @ 1.80Ghz (8 CPUs), ~2.3Ghz 16G RAM. All the codes were written in Python 3.

3.1 Mathematical Model

Before trying to optimize the simulation, it is primordial to build the process from the bottom. It is necessary to create a logic strategy for the sake of reaching the whole terpolymer finished and completed. To illustrate that, [Figure 3](#) is representing the process of terpolymerization at steady state utilizing Monte Carlo model. The model was based in the example proposed in the work of Soares and McKenna³⁴. Table 1 displays the complete mechanism used in the present work.

Table 1: Polymerization Modeling to comonomers

Description	Chemical equations	Rate constants
Initiation (<i>i</i>)	$P_0 + A \rightarrow P_1^A$	k_{iA}
	$P_0 + B \rightarrow P_1^B$	k_{iB}
	$P_0 + C \rightarrow P_1^C$	k_{iC}
Propagation (<i>p</i>)	$P_r^A + A \rightarrow P_{r+1}^A$	k_{pAA}
	$P_r^A + B \rightarrow P_{r+1}^B$	k_{pAB}
	$P_r^A + C \rightarrow P_{r+1}^C$	k_{pAC}
	$P_r^B + A \rightarrow P_{r+1}^A$	k_{pBA}
	$P_r^B + B \rightarrow P_{r+1}^B$	k_{pBB}
	$P_r^B + C \rightarrow P_{r+1}^C$	k_{pBC}
	$P_r^C + A \rightarrow P_{r+1}^A$	k_{pCA}
	$P_r^C + B \rightarrow P_{r+1}^B$	k_{pCB}
	$P_r^C + C \rightarrow P_{r+1}^C$	k_{pCC}
Transfer to monomer (<i>t</i>)	$P_r^A + A \rightarrow P_1^A + D_r$	k_{tAA}
	$P_r^A + B \rightarrow P_1^B + D_r$	k_{tAB}
	$P_r^A + C \rightarrow P_1^C + D_r$	k_{tAC}
	$P_r^B + A \rightarrow P_1^A + D_r$	k_{tBA}
	$P_r^B + B \rightarrow P_1^B + D_r$	k_{tBB}
	$P_r^B + C \rightarrow P_1^C + D_r$	k_{tBC}
	$P_r^C + C \rightarrow P_1^A + D_r$	k_{tCA}

	$P_r^C + C \rightarrow P_1^B + D_r$	k_{tCB}
	$P_r^C + C \rightarrow P_1^C + D_r$	k_{tCC}
Transfer to H_2	$P_r^A + H_2 \rightarrow P_0 + D_r$	k_{tHA}
	$P_r^B + H_2 \rightarrow P_0 + D_r$	k_{tHB}
	$P_r^C + H_2 \rightarrow P_0 + D_r$	k_{tHC}
Transfer to cocatalyst (Al)	$P_r^A + Al \rightarrow P_0 + D_r$	k_{tAlA}
	$P_r^B + Al \rightarrow P_0 + D_r$	k_{tAlB}
	$P_r^C + Al \rightarrow P_0 + D_r$	k_{tAlC}
β -Hybride elimination (td)	$P_r^A \rightarrow P_0 + D_r$	k_{tdA}
	$P_r^B \rightarrow P_0 + D_r$	k_{tdB}
	$P_r^C \rightarrow P_0 + D_r$	k_{tdC}

The three ~~first~~ equations below are extracted from the reactor comonomer molar fractions and reactivity ratios using the Alfrey-Goldfinger equation for polymers.



$$\gamma_A = \left(\frac{f_A}{r_{CA}r_{BA}} + \frac{f_B}{r_{BA}r_{CB}} + \frac{f_C}{r_{CA}r_{BC}} \right) \left(f_A + \frac{f_B}{r_{AB}} + \frac{f_C}{r_{AC}} \right) \quad (1)$$

$$\gamma_B = \left(\frac{f_A}{r_{AB}r_{CA}} + \frac{f_B}{r_{AB}r_{CB}} + \frac{f_C}{r_{CA}r_{BC}} \right) \left(f_B + \frac{f_A}{r_{BA}} + \frac{f_C}{r_{BC}} \right) \quad (2)$$

$$\gamma_C = \left(\frac{f_A}{r_{AC}r_{BA}} + \frac{f_B}{r_{BC}r_{AB}} + \frac{f_C}{r_{AC}r_{BC}} \right) \left(f_C + \frac{f_A}{r_{CA}} + \frac{f_B}{r_{CB}} \right) \quad (3)$$

Reorganizing the last equations, it is possible to have these others from the average comonomer molar fractions in the copolymer considering that $F_x = \gamma_x f_x$ with $x = A, B, C$.

$$F_A = \left(1 + \frac{\gamma_B f_B + \gamma_C f_C}{\gamma_A f_A} \right) \quad (4)$$

$$F_B = \left(1 + \frac{\gamma_A f_A + \gamma_C f_C}{\gamma_B f_B} \right) \quad (5)$$

$$F_C = 1 - F_B - F_A \quad (6)$$

Table 2 shows the first conditions (the second step exhibited in Figure 3). The f represents reactor comonomer molar fraction. The F is originated by the reactor comonomer molar fractions and its reactivity, it is necessary to calculate Pp . The limit to chain transfer reactions is represented by ρt . The number average chain length is represented by r_n .

The m and r insertions can be treated as the copolymerization of comonomers A , B to predict the stereoregularity distribution of the polymer using Bernoullian or Markov models of varying orders, where

$$kpAA = kpBB = kpm \quad (7)$$

$$kpAB = kpBA = kpr \quad (8)$$

The kpm and kpr are the propagation rate constants for meso and racemic insertions.

$$R_{ij} = \frac{k_{pii}}{k_{pij}} \quad i = A, B, C \quad (9)$$

where the active site is represented by P_0 ; P_r^A are the living polymer of chain length r with A as monomer; P_r^B are the living polymer of chain length r with B as monomer; P_r^C are the living polymer of chain length r with C as monomer; D_r is the dead polymer with chain length as r .

Table 2: Simulation's variables and parameters

Parameter	Value
rAB	5.0
rAC	10
rBA	0.2
rBC	2.0
rCA	0.1
rCB	0.5

fA	0.6
fB=fC	0.2
ptA=ptB	0.33
ptC	0.34
F_A	0.9091
F_B	0.0606
F_C	0.0303
r_n	10^3

$$\rho_{pA} = \frac{R_{pA}}{R_{pA} + R_{tA}} = \frac{k_{pAA}[A] + k_{pAB}[B] + k_{pAC}[C]}{k_{pAA}[A] + k_{pAB}[B] + k_{pAC}[C] + K_{TA}} \quad (10)$$

$$\rho_{tA} = 1 - \rho_{pA} \quad (11)$$

where ρ_{tA} and ρ_{pA} are the probabilities of propagation and termination, respectively, for chains terminated in monomer A and K_{TA} is the chain transfer frequency for chains terminated in monomer A. R_{tA} and R_{pA} are the overall rates of propagation and chain transfer for comonomer A, respectively.

The chain length of all intervals is counted. It means that the chain weights of N_{ith} chain length interval $\omega_{MWD}(N_i)$ and the chain weight of N_{ith} molecular fraction interval $\omega_{Fi}(N_i)$ ($i = A, B, C$) are calculated as the following equations:

In the equation 12, $\sum_j R_{tj}$ is the overall rate of chain transfer and the expression itself represents the fraction of chains that undergo chain transfer reactions when their last monomer unit is A.

$$\rho_{tA} = \frac{R_{tA}}{\sum_j R_{tj}} \quad (12)$$

The equation 13 represents the probability of propagation for chains terminated in monomer A.

$$\rho_{pA} = \frac{F_A r_n}{F_A r_n + \rho_{tA}} \quad (13)$$

The equation 14 and 15 below represents respectively the chain weight of N_{ith} chain length interval; $\omega_{MWD}(N_i)$. And the chain weight of N_{ith} molar fraction interval; $\omega_{Fi}(N_i)$.

$$\omega_{MWD}(N_i) = \sum n(k), \text{ for } N_i \leq n(k) \leq N_{i+1} \text{ } i=A, B, C \quad (14)$$

$$\omega_{Fi}(N_i) = \sum n(k), \text{ for } N_i \leq F_i(k) \leq N_{i+1} \text{ } i=A, B, C \quad (15)$$

The chain weights of all intervals are scaled by their sum as follows and then the MWD and CCD can be calculated to the simulation's conclusion.

$$mwd(i) = \frac{w_{MWD}(N_i)}{\sum_{i=1}^{N_{MWD}}(N_i)} \quad i = A, B, C \quad (16)$$

$$ccd_{F_i}(i) = \frac{w_{F_i}(N_i)}{\sum_{i=1}^{N_{CCD}} w_{F_i}(N_i)} \quad i = A, B, C \quad (17)$$

In equation 17 the $ccd_{F_i}(i)$ calculates the CCD for each average comonomer molar fraction in the copolymer, the same for equation 16 but for MWD.

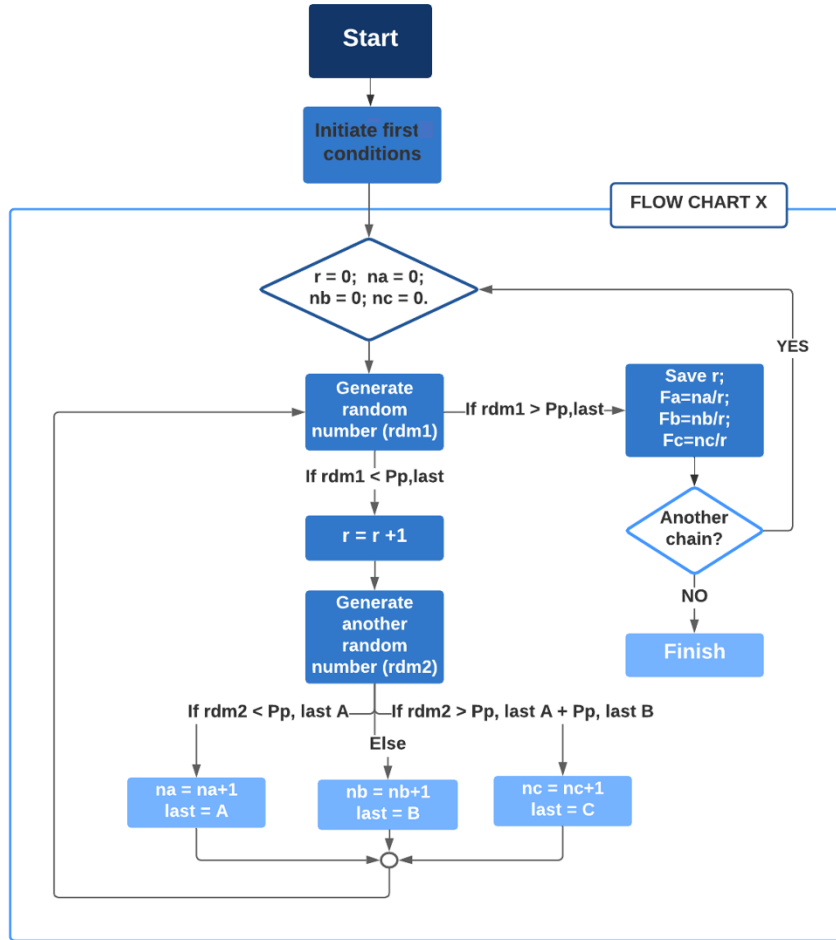


Figure 3: Polymerization's flowchart.

The different monomers are represented by letters A, B and C; “Pp” is the probability of propagation; “ult” referred to the last monomer unit added to the polymer crescent chain (A, B or C); “rdm1” and “rdm2” are random numbers that belongs in the range [0,1); The actual polymeric chain length appear as “r”; The variables “na”, “nb” and “nc” are the numbers of times the monomers A, B and C, respectively, are inserted in the crescent polymeric chain.

After the simulation, represented by the algorithm exhibited on flowchart on [Figure 3](#), the molar fractions of the comonomers are saved in the variables “Fa”, “Fb” and “Fc”, and the array responsible to carry the total chain length (D).

Once these arrays are randomly distributed, it is necessary to ordinate them. Part of this study is dedicated to the research for sorting methods that could suit properly the double and directly dependent ordination that this terpolymer simulation requires. The D array will demand crescent sorting and each position attributed to this will need to be followed by the vectors Fa, Fb and Fc as illustrated below in [Figure 4](#).

D	Fa	Fb	Fc
7	0.5	0.33	0.17
1	0.1	0.3	0.6
...	...	...	...
2	0.3	0.7	0
3	0.2	0.5	0.3

D	Fa	Fb	Fc
1	0.1	0.3	0.6
2	0.3	0.7	0
3	0.2	0.5	0.3
...	...	...	...
7	0.5	0.33	0.17

Figure 4: Sorting process.

3.3 Optimization Processes

Many resources can be used to improve the time that the algorithm spends executing each program line written to simulate the polymer. The strategies researched evolve to apply techniques of parallelization through two Python libraries: Joblib and Numba.

To make the comparison between these two possibilities more reproducible, the simulation was followed for a same sorting algorithm in order to compare the methods in a whole program. Even though the differences will be measured by the simulation time only, which gives less deviation because it is a way to put both strategies face to face. It was defined that the whole program should be ran 10 times and its computational time average will be the result considered.

3.3.1 Joblib Parallelization

Joblib is a wedge of Python library, it permits processing command lines simultaneously splitting tasks in the CPU existents – the virtual or the machinery presented in use. This strategy is also named for multiprocessing or parallelization and allows a computational valuable gain to polymerization. The idea is utilizing this path to possibilities that each monomer be created independently in a parallel manner and posteriorly all of this be united to generate and increment the final polymer. In the case of existing four CPUs on the computer processor, for example, the program will have its time reduced by the fourth part if the artifice of parallelism was implemented in the right way. This example is illustrated on Figure 2.

3.3.2 Numba

Numba is nothing but an existent compiler originated from the Python language. It converts all written code in an intermediate representation, which is called “no python mode”. Through this the program can be executed without the pre settled global block of the algorithm. This accelerator also permits the parallelization of the simulation, like [Erro! Fonte de referência não encontrada. Figure 2](#) shown and explained previously for Joblib. It is an alternative and concurrent resource to the exhibited in last paragraph and [its turns possible the comparison verifying its computational times spent in either processing facing both.](#)

3.4 Sorting Algorithms

There are many different algorithms that can be used to sort exit arrays that came out of the terpolymer simulation. This sorting process is essential considering that it is an element that makes possible the comparison with the model chosen as example and its final result.

According to the Big-O notation the average complexity for the runtime of Bubble, Insertion and Selection sort are the same ($O(n^2)$) while for Timsort it rates $O(n \log n)$ — very low. For the first three algorithms, as much as the array gets bigger, the execution time grows for both in the same rate and the difference is in the steps taken until a full sorted array. Bubble algorithm swaps an unsorted array moving one element to extremity while Insertion makes this swap into an already sorted array and does not move elements through. Bubble must check to make sure it is sorted when insertion doesn't need to. Selection is only good for small arrays, but in all occasions still performing less swaps than Insertion. Bubble is known as the worst sorting algorithms among all of the chosen ones; it is not a practical sorting algorithm, but for small arrays it works with an efficiency close to the others which have same Big-O notation. That happens because it is easy for the algorithm to recognize an almost or completely sorted array.

It is important to mention that each sorting algorithm analyzed executes individual procedures to implement the sorting and that steps taken by the code are directly related with the final computational time at the end of the program. For that reason, this study is so relevant to the whole research for itself.

To improve comparison, it was defined that each algorithm should be tested 10 times too and its computational time average of it obtained, it will be the value used as result. Besides that, only one same array randomly generated was conducted through the research. All the sorting algorithms were applied on this unique array in order to promote an equal analysis when searching for the best sorting algorithm.

3.4.1 Bubble sorting

The Bubble algorithm starts the sorting procedure from the beginning of the array. Then, a decision must be made between swapping positions if the next number is smaller than the previous or not, in case it is larger. That can be observed in Figure 5. This process is repeated until the last element of the array is reached. Subsequently, the algorithm checks if any swap still needs to be made. If yes, the process repeats until all the elements of the array are ordered correctly.

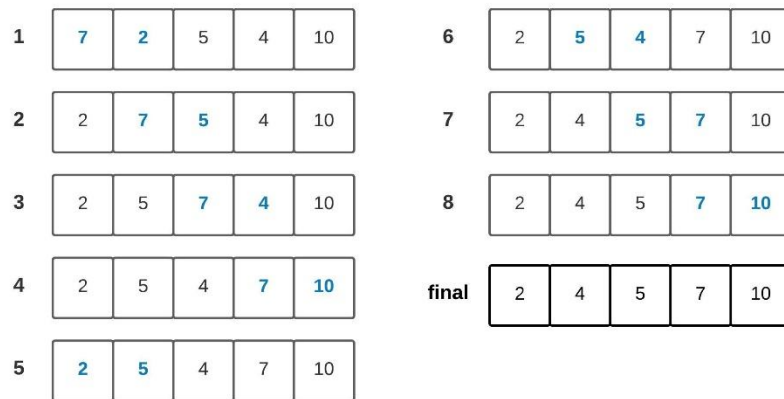


Figure 5: Bubble sorting algorithm.

3.4.2 Insertion Sorting

The Insertion algorithm also starts sorting from the first element of the array as showed in Figure 6. After, it goes through the array making the decision bounded by: if there is no smaller element to swap in the array, proceed; if there is a lack of order, insert the element in the right place. Repeating this procedure to the sequential numbers, at the end of the array the sort is over.

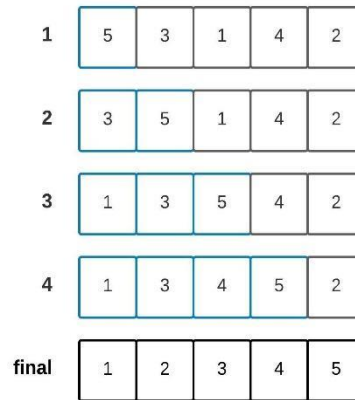


Figure 6: Insertion sorting algorithm.

3.4.3 Selection Sorting

Another algorithm that was tested is the Selection. It starts sorting from the first element of the array. The entire process is demonstrated on Figure 7. The first element is saved and from this information the analysis sur all the array is made searching for a smaller number than the recorded one. If it happens, the primary number is substituted by the found. When finished searching for all the array, the smallest element would be selected, and it's moved for the first position. This procedure continues this along the array until reach a complete **sorting**.

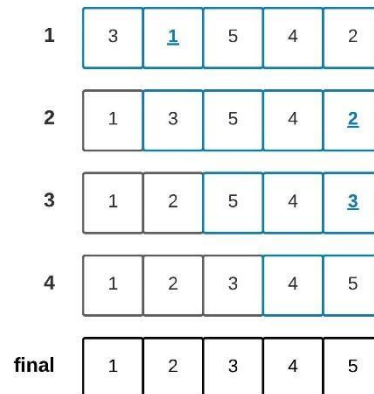


Figure 7: Selection sorting algorithm.

3.4.4 Timsort

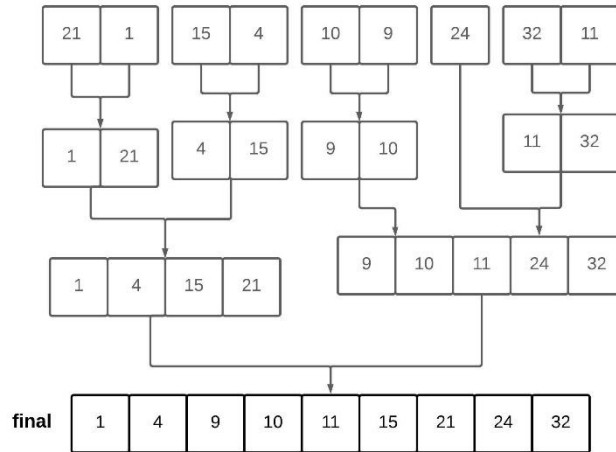


Figure 8: Timsort algorithm.

Finally, the Timsort algorithm divides the main array in sub-arrays (fragments with 32 to 65 elements have been shown to perform best). Next, each sub-array is sorted using the Insertion algorithm. Afterwards, these sub-arrays are combined using the Merge algorithm – process exhibited in Figure 8 from the beginning. It consists in separate two any of these sub-arrays already sorted, then make a new sorted array (final array S) from both. It is necessary to first mark the current position of both arrays. Next, compare the elements of each a move the smallest to the sorted array (S). Repeat the procedure moving the mark to the succeeding elements of the array to the end of the sorted one (S) if one chosen array has different length from the other.

4. Results and Discussion

The comparison between the sorting methods was made utilizing three distinct maximum chain lengths (N): 100; 500; 10000. The objective was to measure the efficiency based on the time spent to organize the arrays that are disposed similarly from arrays when leaving simulations, observing its dependence with the N to get into some explanations.

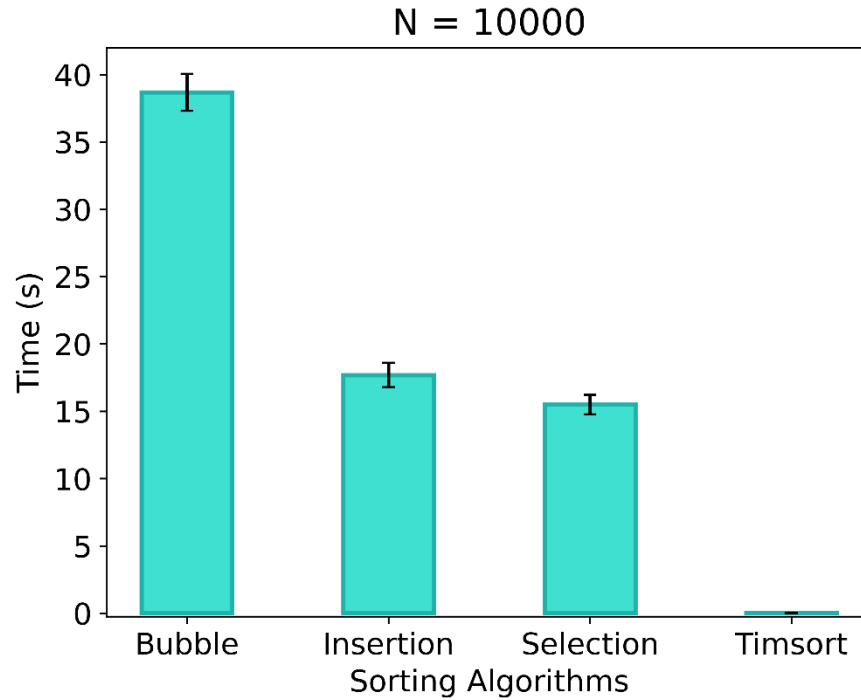


Figure 9: Sorting Algorithms $N=10000$

Figure 10 exhibits the sorting average times calculated after 10 times run of each method for a same array with N equal to 10.000. The Standards Deviations (SD) are available in the figure itself and considering it, there's no great difference between Selection and Insertion sorting algorithms. The SD for Timsort is so small that is not even identified in the bar graph. It can be noticed that the time spent on the algorithm itself is close to zero, which explain why the SD shows insignificant values.

Figure 10 also shows that the worst sorting method for the present scenario in the research is the Bubble Sort. This result was already expected since the bibliography contains articles mentioning that this algorithm works best with short arrays. The Big-O notation previously pointed out that Insertion, Selection and Bubble sorting algorithms have higher complexity average then Timsort. What distinguish these three methods is the way the steps are taken to have an array completely sorted. For sorted arrays, the algorithms can quickly identify it and its complexity pass to only $O(n)$. Depending on how tricky the array is sorted, Bubble can pass along the array several times until final confirm the conclusion of proposed task.

The results of the comparison of sorting algorithms reached using the N equal to 100 were inconclusive and for that reason the figure was not disposable in this article. The sorting times do not present any disparity at all, considering their SDs. The three worst algorithms (Selection, Insertion and Bubble), at this maximum chain length, reached the same efficiency of Timsort. It crops up the other algorithms conducted well the sorting in small arrays while Timsort uses too many effort to solve this simple problem. In other words, Timsort raises its time concomitantly the further algorithms reduce it. Explaining this equally of times to run the procedure for compact values of N .

For that reason, the simulation was tested with a N equal to 500. It is a chain length longer than 100 and it is still small enough when facing the biggest used in the research N (10.000). This extra N trial was employed to provide an intermediate analysis and bring some discussion about the algorithm's behavior when facing not so long arrays yet not too small either.

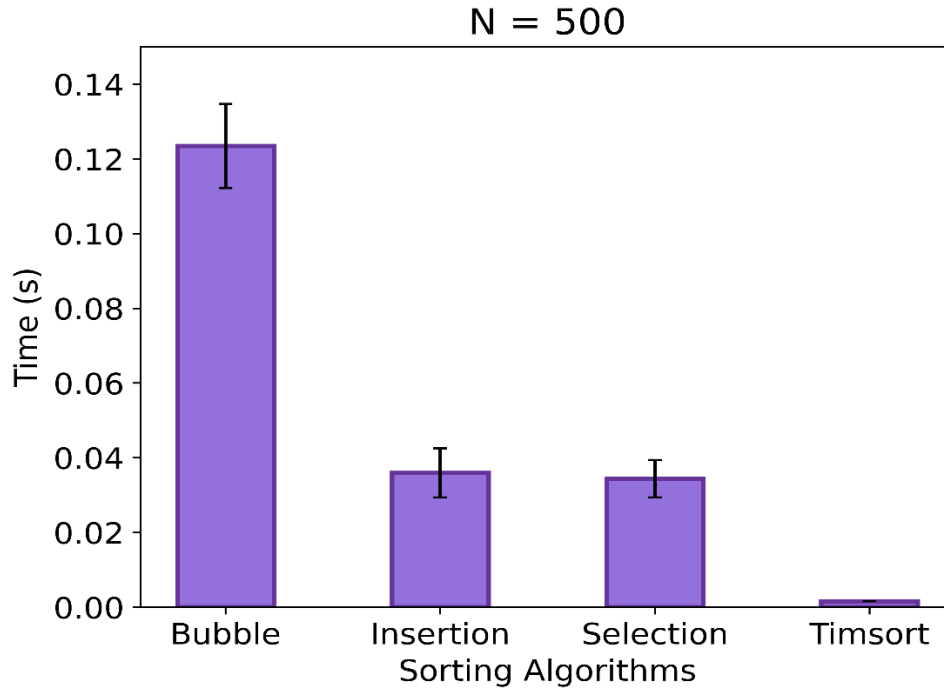


Figure 10: Sorting Algorithms N=500

Under 100 elements in the array there is no difference among the types of methods, even knowing that Timsort reduces its potency when it is a short sorting. Apart from that, when looking at the Figure 11, the Timsort highlights again. It stands out the very best sorting algorithm among others. In second place it is the Selection and Insertion. Both according to the Big O, they work faster than Bubble and it can be noticed in the research either, it accentuates if the array gets bigger. For the chain length equal to 100, there was no distinguish midst the four sorting algorithms. While for 500 as maximum length its observed that clearly it has a slowest algorithm – Bubble - and a fastest one – Timsort.

Having the best sorting method chosen, it is time to discuss why the N equal to 10000 was the selected for this research. It had to respect two particularities: N couldn't take too much time to finish the simulation and sorting, because it would be run several times during the tests; it must fit the model utilized in the studied case, guaranteeing a trustworthy analysis.

The simulations started by running with a N size of 1000. The computational time was long, but the final figure result couldn't ensure the correspondence with the model, the experimental process wasn't verified. If the N size raises, the time gets higher while the adjusting to the model increases. Reaching the 10000 (Figure 12) mark the figure seems very likely the one presented in Soares and McKenna¹⁹. To ensure that N value was the optimum, it was running in N equal to 20000 (Figure 13) to check if there was a big difference between figures or spent time.

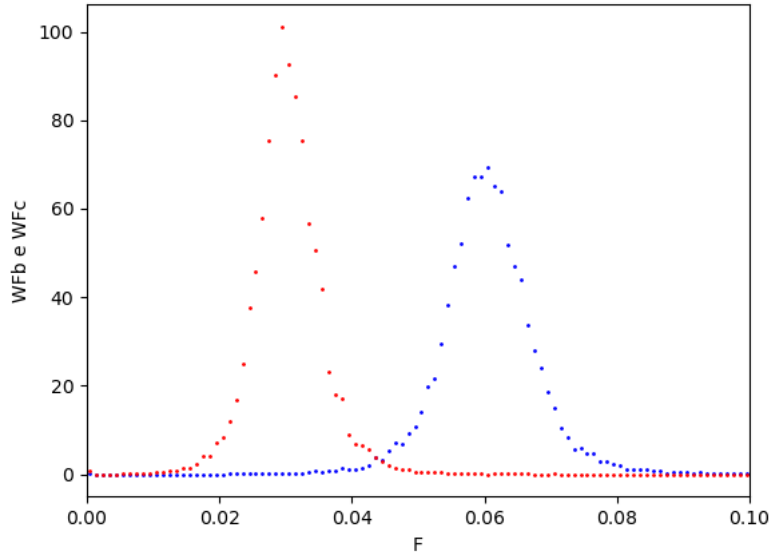


Figure 11: Weight Mass Distribution with N equal to 10000

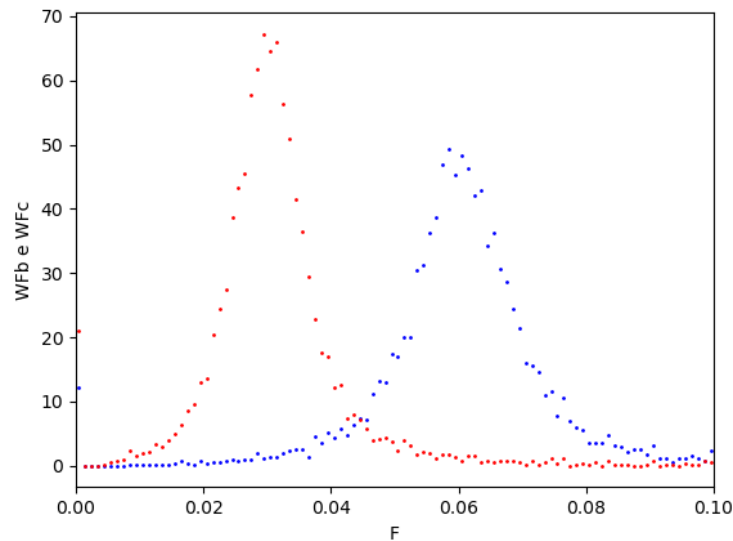


Figure 12: Weight Mass Distribution with N equal to 20000

In Figure 12, it is disposed the Molecular Weight Distribution (MWD) for the monomers C and B with the maximum chain length settled in 10000. There is not much disparity when facing the curves in Figure 13, that shows up the same vision however with N equal to 20000 (more data available in graph). The issue of using the last value of N presented it is the time spent during the entire simulation program. Almost 7.65 times higher than the time spent with N equal to 10000. Balancing these both sides, it was decided to execute all the correlations of simulations using this optimum value of N in the interest of choosing a number that warranty the quality of the research.

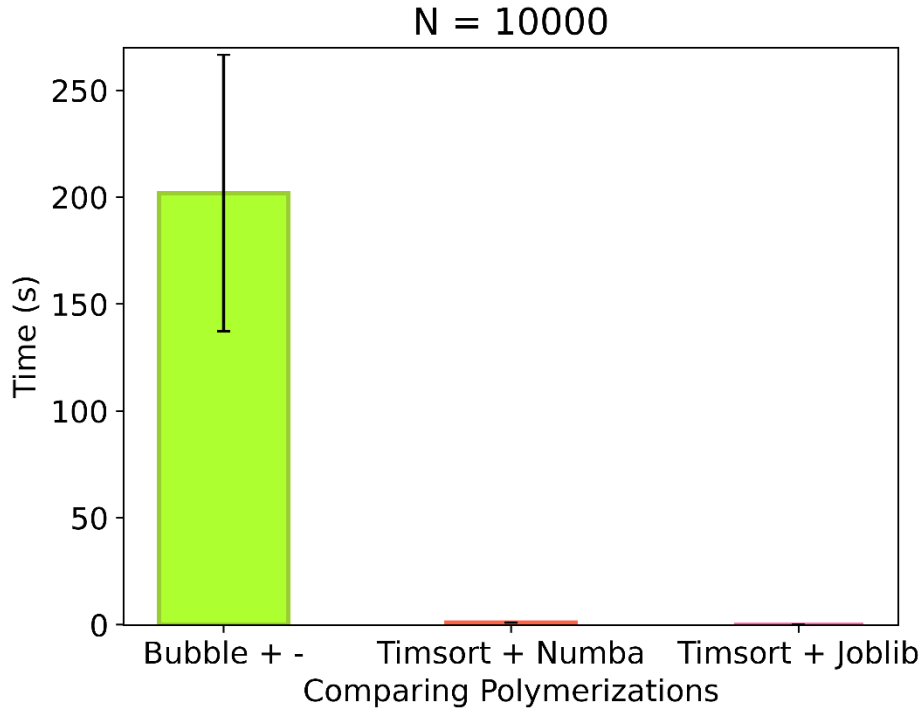


Figure 13: Total Simulation N equal to 10000

After visualizing the best sorting algorithm in action, it was employed in addition to optimization techniques that could reduce even more the computational time. The objective is to find the best combination between simulation and sorting to **final get into** the most improved terpolymerization **ran** of all tested.

In Figure 14 is compared the worst sorting algorithm tested with no optimization process allied to accelerate the simulation (green bar) with the best sorting algorithm tested (Timsort) joined with two different optimization resources, Numba and Joblib. The middle bar shows the result of a non-python mode which is the characteristic of this method, it handles well in terms of time reduction. However, the attention turns completely for the last bar, so small that it is difficult to measure. Timsort combined with Joblib is the winner of the code enhancement race. These two methods together make the whole program works in $0.053s \pm 0.03$. The green bar time is in average 201.91 ± 64.70 . It means that considering the worst-case scenario, the effectiveness of the best time founded is 99.94%. In other words, this percentage represents the time saved in the **procedure**.

The average time spent during the simulation with no improvements is about 0.7% of the whole program by itself, using Bubble as sorting algorithm along the code. While betting the Joblib as an optimization factor, it almost does not take any time to generates the terpolymer desired. It all culminated that the simulation acceleration is due almost entirely by the sorting algorithm chosen, in this case Timsort. That is the importance of studying hardly manners to sort fast, utilizing less steps and the lowest complexity possible. Combining a parallel process to execute the simulation in conjunction of a powerful sorting algorithm led the research to this positive result.

The best part of this results is not the number properly, but the possibilities of employment these two-method allied – or not - in any other future research. Joblib can be applied in other simulations, with more complexity or utilizing bigger N values and it will still work fine because the task division will continue being executed and time economized. At the same time, Timsort can be utilized in other MC simulations, since the method creates arrays completely randomly. Even so, it is just one of many **possibilists** in view of the fact that this feature could be used in any code that needs a sorting algorithm to operate.

Future next steps to continue this research is to study the implementation of the dynamic state and even in the steady state if it is possible to explore the simulation utilizing bigger maximum number of chains. Bringing all the information already acquire from this present study, the succeeding work will be **benefit** and **start in advance**.

5. Conclusion

The present study successfully performed the steady-state MC simulation of terpolymerization. Different scenarios were analysed using different sorting algorithms (Bubble, Selection, Insertion, and timsort), and also different tools to generate the polymeric chains (Joblib and Numba). The worst scenario simulated was the one utilizing Bubble sorting with no simulation speed method beating almost 202 seconds to run the entire program. The research showed that Numba is a great optimization method to utilized also, the effectiveness was very similar to the Joblib. The studied work categorized the sorting algorithms explaining why Bubble **if fine** for small arrays ~~mostly~~ and the differences between all sorting strategies and times spent. To summarize, **at** the end the best sorting method was Timsort allied with the efficiency of Joblib parallelization which reduced the time spent in the simulation by 99.94% acquiring a good result and improvement to the polymerization.

6. Bibliography

- (1) Vicente Canevarolo Jr., S. *CIÊNCIA DOS POLÍMEROS* (2010, ARTLIBER EDITORA LTDA); ARTLIBER EDITORA LTDA.
- (2) Siqueira, J. R.; Oliveira, O. N. Carbon-Based Nanomaterials. In *Nanostructures*; Elsevier Inc., 2017; pp 233–249. <https://doi.org/10.1016/B978-0-323-49782-4.00009-7>.
- (3) Lopes, C. M. A.; Gonçalves, D. P.; De Melo, F. C. L. *RESISTÊNCIA BALÍSTICA DE COMPÓSITOS POLIMÉRICOS LAMINADOS*.
- (4) Maitz, M. F. Applications of Synthetic Polymers in Clinical Medicine. *Biosurface and Biotribology* **2015**, 1 (3), 161–176. <https://doi.org/10.1016/j.bsbt.2015.08.002>.
- (5) William D. Callister, J.; Rethwisch, D. G. Phase Transformations: Development of Microstructure and Alternation of Mechanical Properties. *Mater. Sci. Eng. An Introd.* **2013**, 356–407.
- (6) Pioli Vieira, R.; Maria Ferrareso Lona, L. Simulation of Temperature Effect on the Structure Control of Polystyrene Obtained by Atom-Transfer Radical Polymerization. **2016**, 26 (4),

- 313–319. <https://doi.org/10.1590/0104-1428.2376>.
- (7) Pintos, E.; Sarmoria, C.; Brandolin, A.; Asteasuain, M. Modeling of RAFT Polymerization Processes Using an Efficient Monte Carlo Algorithm in Julia. *Ind. Eng. Chem. Res.* **2016**, *55* (31), 8534–8547. <https://doi.org/10.1021/acs.iecr.6b01639>.
 - (8) Segovia-Hernández, J. G.; Hernández, S.; Bonilla Petriciolet, A. Reactive Distillation: A Review of Optimal Design Using Deterministic and Stochastic Techniques. *Chemical Engineering and Processing: Process Intensification*. Elsevier November 1, 2015, pp 134–143. <https://doi.org/10.1016/j.cep.2015.09.004>.
 - (9) Kumar, A.; Gupta, S. K.; Madan, S.; Shah, N. G.; Gupta, S. K. Solution of Final Stages of Polyethylene Terephthalate Reactors Using Orthogonal Collocation Technique. *Polym. Eng. Sci.* **1984**, *24* (3), 194–204. <https://doi.org/10.1002/pen.760240306>.
 - (10) Gonzalez-romero, V. M.; Rodriguez, B. E. Orthogonal Collocation Solution for the Free Radical Polymerization with Diffusional Limitations. *Chem. Eng. Commun.* **1987**, *59* (1–6), 185–200. <https://doi.org/10.1080/00986448708911994>.
 - (11) Saliakas, V.; Chatzidoukas, C.; Krallis, A.; Meimaroglou, D.; Kiparissides, C. Dynamic Optimization of Molecular Weight Distribution Using Orthogonal Collocation on Finite Elements and Fixed Pivot Methods: An Experimental and Theoretical Investigation. *Macromol. React. Eng.* **2007**, *1* (1), 119–136. <https://doi.org/10.1002/mren.200600015>.
 - (12) Galvan, R.; Tirrell, M. Orthogonal Collocation Applied to Analysis of Heterogeneous Ziegler-Natta Polymerization. *Comput. Chem. Eng.* **1986**, *10* (1), 77–85. [https://doi.org/10.1016/0098-1354\(86\)85048-7](https://doi.org/10.1016/0098-1354(86)85048-7).
 - (13) Ma, Y.; Chen, X.; Eason, J. P.; Biegler, L. T. Dynamic Optimization for Grade Transition Processes Using Orthogonal Collocation on Molecular Weight Distribution. *AIChE J.* **2019**, *65* (4), 1198–1210. <https://doi.org/10.1002/aic.16524>.
 - (14) Saldívar-Guerra, E. Numerical Techniques for the Solution of the Molecular Weight Distribution in Polymerization Mechanisms, State of the Art. *Macromol. React. Eng.* **2020**, *14* (4), 1–21. <https://doi.org/10.1002/mren.202000010>.
 - (15) Asteasuain, M.; Brandolin, A. Optimal Operation of Ethylene Polymerization Reactors for Tailored Molecular Weight Distribution. *J. Appl. Polym. Sci.* **2007**, *105* (5), 2621–2630. <https://doi.org/10.1002/app.26295>.
 - (16) Brandolin, A.; Sarmoria, C.; Lopez-Rodriguez, A.; Whiteley, K. S.; Del Amo Fernandez, B. Prediction of Molecular Weight Distributions by Probability Generating Functions. Application to Industrial Autoclave Reactors for High Pressure Polymerization of Ethylene and Ethylene-Vinyl Acetate. *Polym. Eng. Sci.* **2001**, *41* (8), 1413–1426. <https://doi.org/10.1002/pen.10841>.
 - (17) Asteasuain, M.; Sarmoria, C.; Brandolin, A. Controlled Rheology of Polypropylene: Modeling of Molecular Weight Distributions. *J. Appl. Polym. Sci.* **2003**, *88* (7), 1676–1685. <https://doi.org/10.1002/app.11823>.

- (18) Castor, C. A.; Sarmoria, C.; Asteasuain, M.; Brandolin, A.; Pinto, J. C. Mathematical Modeling of Molecular Weight Distributions in Vinyl Chloride Suspension Polymerizations Performed with a Bifunctional Initiator through Probability Generating Functions. *Macromol. Theory Simulations* **2014**, *23* (8), 500–522. <https://doi.org/10.1002/mats.201400038>.
- (19) Asteasuain, M.; Brandolin, A.; Sarmoria, C. Molecular Weight Distributions in Styrene Polymerization with Asymmetric Bifunctional Initiators. *Polymer (Guildf)*. **2004**, *45* (1), 321–335. <https://doi.org/10.1016/j.polymer.2003.10.083>.
- (20) Rego, A. S. C.; Brandão, A. L. T. General Method for Speeding Up Kinetic Monte Carlo Simulations. *Ind. Eng. Chem. Res.* **2020**, *59* (19), 9034–9042. <https://doi.org/10.1021/acs.iecr.0c01069>.
- (21) Gillespie, D. T. A General Method for Numerically Simulating the Stochastic Time Evolution of Coupled Chemical Reactions. *J. Comput. Phys.* **1976**, *22* (4), 403–434. [https://doi.org/10.1016/0021-9991\(76\)90041-3](https://doi.org/10.1016/0021-9991(76)90041-3).
- (22) Gao, H.; Oakley, L. H.; Konstantinov, I. A.; Arturo, S. G.; Broadbelt, L. J. Acceleration of Kinetic Monte Carlo Method for the Simulation of Free Radical Copolymerization through Scaling. *Ind. Eng. Chem. Res.* **2015**, *54* (48), 11975–11985. <https://doi.org/10.1021/acs.iecr.5b03198>.
- (23) Weng, J.; Chen, X.; Yao, Z.; Biegler, L. T. Parallel Monte Carlo Simulation of Molecular Weight Distribution and Chemical Composition Distribution for Copolymerization on a Graphics Processing Unit Platform. *Macromol. Theory Simulations* **2015**, *24* (5), 521–536. <https://doi.org/10.1002/mats.201500021>.
- (24) Fang, C.; Zhang, J.; Chen, X.; Weng, G. J. Calculating the Electrical Conductivity of Graphene Nanoplatelet Polymer Composites by a Monte Carlo Method. <https://doi.org/10.3390/nano10061129>.
- (25) Demirel Özçam, D.; Teymour, F. Chain-by-Chain Monte Carlo Simulation: A Novel Hybrid Method for Modeling Polymerization. Part I. Linear Controlled Radical Polymerization Systems. *Macromol. React. Eng.* **2017**, *11* (1), 1600042. <https://doi.org/10.1002/mren.201600042>.
- (26) Mohammadi, Y.; Saeb, M. R.; Penlidis, A.; Jabbari, E.; Zinck, P.; Stadler, F. J.; Matyjaszewski, K. Intelligent Monte Carlo: A New Paradigm for Inverse Polymerization Engineering. *Macromol. Theory Simulations* **2018**, *27* (3), 1700106. <https://doi.org/10.1002/mats.201700106>.
- (27) Drache, M.; Brandl, K.; Reinhardt, R.; Beuermann, S. Ab Initio Kinetic Monte Carlo Simulation of Seeded Emulsion Polymerizations of Styrene. *Phys. Chem. Chem. Phys.* **2018**, *20* (16), 10796–10805. <https://doi.org/10.1039/c7cp07768h>.
- (28) Hsissou, R.; Abbout, S.; Seghiri, R.; Rehioui, M.; Berisha, A.; Erramli, H.; Assouag, M.; Elharfi, A. Evaluation of Corrosion Inhibition Performance of Phosphorus Polymer for Carbon Steel in [1 M] HCl: Computational Studies (DFT, MC and MD Simulations). *J.*

- Mater. Res. Technol.* **2020**, 9 (3), 2691–2703. <https://doi.org/10.1016/j.jmrt.2020.01.002>.
- (29) Elkholy, A. E.; El-Taib Heakal, F.; Rashad, A. M.; Zakaria, K. Monte Carlo Simulation for Guar and Xanthan Gums as Green Scale Inhibitors. *J. Pet. Sci. Eng.* **2018**, 166, 263–273. <https://doi.org/10.1016/j.petrol.2018.03.019>.
 - (30) Detcheverry, F. A.; Pike, D. Q.; Nealey, P. F.; Müller, M.; De Pablo, J. J. Monte Carlo Simulation of Coarse Grain Polymeric Systems. *Phys. Rev. Lett.* **2009**, 102 (19), 197801. <https://doi.org/10.1103/PhysRevLett.102.197801>.
 - (31) Román, S.; Lund, F.; Bustos, J.; Palza, H. About the Relevance of Waviness, Agglomeration, and Strain on the Electrical Behavior of Polymer Composites Filled with Carbon Nanotubes Evaluated by a Monte-Carlo Simulation. *Mater. Res. Express* **2018**, 5 (1), 015044. <https://doi.org/10.1088/2053-1591/aaa531>.
 - (32) Nie, Y.; Liu, Y.; Liu, R.; Zhou, Z.; Hao, T. Dynamic Monte Carlo Simulations of Competition in Crystallization of Mixed Polymers Grafted on a Substrate. *J. Polym. Sci. Part B Polym. Phys.* **2019**, 57 (2), 89–97. <https://doi.org/10.1002/polb.24757>.
 - (33) Lai, P. Y.; Binder, K. Structure and Dynamics of Grafted Polymer Layers: A Monte Carlo Simulation. *J. Chem. Phys.* **1991**, 95 (12), 9288–9299. <https://doi.org/10.1063/1.461158>.
 - (34) Soares, J. B. P.; McKenna, T. F. L. Polyolefin Microstructural Modeling. *Polyolefin React. Eng.* **2012**, 187–269. <https://doi.org/10.1002/9783527646944.ch6>.
 - (35) Ramos, J.; Peristeras, L. D.; Theodorou, D. N. Monte Carlo Simulation of Short Chain Branched Polyolefins in the Molten State. *Macromolecules* **2007**, 40 (26), 9640–9650. <https://doi.org/10.1021/ma071615k>.
 - (36) Inkson, N. J.; Das, C.; Read, D. J. Monte Carlo Simulation for the Structure of Polyolefins Made with Two Metallocene Catalysts in a Batch Reactor. *Macromolecules* **2006**, 39 (14), 4920–4931. <https://doi.org/10.1021/ma060654d>.
 - (37) Brandão, A. L. T.; Soares, J. B. P.; Pinto, J. C.; Alberton, A. L. When Polymer Reaction Engineers Play Dice: Applications of Monte Carlo Models in PRE. *Macromolecular Reaction Engineering*. Wiley-VCH Verlag June 1, 2015, pp 141–185. <https://doi.org/10.1002/mren.201500020>.
 - (38) Zhao, Y. R.; Buren, B. D.; Puskas, J. E.; McAuley, K. B. A Simple Monte Carlo Method for Modeling Arborescent Polymer Production in Continuous Stirred Tank Reactor. **2018**, 1800020, 1–11. <https://doi.org/10.1002/mren.201800020>.
 - (39) Zhao, Y. R.; Buren, B. D.; Puskas, J. E.; McAuley, K. B. A Simple Monte Carlo Method for Modeling Arborescent Polymer Production in Continuous Stirred Tank Reactor. *Macromol. React. Eng.* **2018**, 12 (5), 1800020. <https://doi.org/10.1002/mren.201800020>.
 - (40) Szymanski, R.; Sosnowski, S. Cumulative Steady State Monte Carlo Method for Processes with Exchange between Reactive Species. Case of Controlled Radical Copolymerization. *Chem. Eng. J.* **2019**, 370, 432–443. <https://doi.org/10.1016/j.cej.2019.03.184>.

- (41) Rego, A. S. C.; Brandã, A. L. T. General Method for Speeding Up Kinetic Monte Carlo Simulations. **2020**. <https://doi.org/10.1021/acs.iecr.0c01069>.
- (42) Pournemat, A.; Wilhelm, F.; Haußmann, J.; Vierrath, S.; Thiele, S.; Scholta, J. A Steady-State Monte Carlo Study on the Effect of Structural and Operating Parameters on Liquid Water Distribution within the Microporous Layers and the Catalyst Layers of PEM Fuel Cells. *J. Electrochem. Soc.* **2018**, *165* (13), F1092–F1097. <https://doi.org/10.1149/2.0861813jes>.
- (43) Brandão, A. L. T.; Soares, J. B. P.; Pinto, J. C.; Alberton, A. L. Comparison of Different Dynamic Monte Carlo Methods for the Simulation of Olefin Polymerization. *Macromol. Symp.* **2016**, *360* (1), 160–178. <https://doi.org/10.1002/masy.201500111>.